

Linguagem SQL

DDL, DML e DQL na prática

Unidade 5 · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Nesta aula

- **SQL**: uma linguagem, quatro grupos — **DDL, DML, DQL, DCL**
- **DDL** — criar e alterar a estrutura (tabelas, chaves, índices, views)
- **DML** — inserir, atualizar e apagar dados
- **DQL** — consultar (`SELECT`): filtros, junções, agregações, subconsultas
- **Portabilidade** Oracle / PostgreSQL / MySQL



Todos os exemplos saem do **Roteiro Prático de SQL** e rodam na **VM LabDatabase**.

O domínio de exemplo (acadêmico)

Um mini sistema acadêmico — o mesmo do Roteiro Prático:

- **ALUNOS, PROFESSORES, DISCIPLINAS**
- **OFERTAS** (uma disciplina, um professor, um horário)
- **ALUNOS_OFERTAS** (matrícula do aluno numa oferta — tabela associativa M:N)
- **TELEFONES_ALUNOS** (telefones de um aluno — 1:N)

💡 Na Parte 3 o roteiro troca para um domínio de **vendas** (clientes, produtos, pedidos) para praticar consultas mais ricas.

Os quatro grupos da SQL

Grupo	Para quê	Comandos
DDL — <i>Definition</i>	estrutura	CREATE , ALTER , DROP
DML — <i>Manipulation</i>	dados	INSERT , UPDATE , DELETE
DQL — <i>Query</i>	consulta	SELECT
DCL — <i>Control</i>	permissões	GRANT , REVOKE

(há ainda o **TCL** — `COMMIT` , `ROLLBACK` , `SAVEPOINT` — visto na Unidade 6.)

DDL — definindo a estrutura

CREATE TABLE

```
CREATE TABLE ALUNOS (  
    MATRICULA      NUMERIC      NOT NULL,  
    NOME           VARCHAR2(100) NOT NULL,  
    DATA_NASCIMENTO DATE       NOT NULL  
);  
  
CREATE TABLE DISCIPLINAS (  
    CODIGO_DISCIPLINA NUMERIC      NOT NULL,  
    NOME_DISCIPLINA   VARCHAR2(100) NOT NULL,  
    CARGA_HORARIA     NUMERIC(3)   NOT NULL,  
    EMENTA            VARCHAR2(4000) NOT NULL,  
    CODIGO_DISCIPLINA_DEPENDENCIA NUMERIC -- auto-relacionamento  
);
```

`NOT NULL` é uma **restrição de integridade**: o campo é obrigatório.

Tipos de dados mais comuns

Categoria	Oracle	PostgreSQL / MySQL
Texto	<code>VARCHAR2(n)</code>	<code>VARCHAR(n)</code>
Inteiro	<code>NUMERIC</code> / <code>NUMBER</code>	<code>INTEGER</code> / <code>INT</code>
Decimal	<code>NUMBER(p,s)</code>	<code>NUMERIC(p,s)</code>
Data	<code>DATE</code>	<code>DATE</code>
Data + hora	<code>TIMESTAMP</code>	<code>TIMESTAMP</code>



Escolha o tipo pelo **significado** do dado — não guarde data como texto nem dinheiro como `float`.

ALTER TABLE — evoluindo a estrutura

```
-- adicionar / modificar coluna
ALTER TABLE ALUNOS ADD EMAIL VARCHAR2(200);
ALTER TABLE ALUNOS MODIFY EMAIL VARCHAR2(250);
ALTER TABLE OFERTAS MODIFY DATA_CRIACAO DATE DEFAULT SYSDATE NOT NULL;

-- renomear tabela e coluna
ALTER TABLE TELEFONES RENAME TO TELEFONES_ALUNOS;
ALTER TABLE ALUNOS RENAME COLUMN MATRICULA TO MATRICULA_ALUNO;

-- remover coluna
ALTER TABLE ALUNOS DROP COLUMN EMAIL;
```

`DROP COLUMN` apaga os dados daquela coluna — é irreversível sem backup.

Chaves: PRIMARY KEY e FOREIGN KEY

```
-- chave primária (simples e composta)
ALTER TABLE ALUNOS      ADD PRIMARY KEY (MATRICULA_ALUNO);
ALTER TABLE ALUNOS_OFERTAS ADD PRIMARY KEY (MATRICULA_ALUNO, CODIGO_OFERTA);

-- chave estrangeira
ALTER TABLE OFERTAS
  ADD CONSTRAINT OFERTAS_PROFESSOR_FK
    FOREIGN KEY (MATRICULA_PROFESSOR)
    REFERENCES PROFESSORES (MATRICULA_PROFESSOR);
```

A **FK** garante a **integridade referencial**: não existe oferta apontando para um professor inexistente.

Sequences e Índices

```
-- sequência para gerar códigos automáticos (Oracle)
CREATE SEQUENCE DISCIPLINAS_SEQ;
INSERT INTO DISCIPLINAS VALUES (DISCIPLINAS_SEQ.NEXTVAL, ...);

-- índice para acelerar buscas por nome
CREATE INDEX ALUNOS_NOME_IDX ON ALUNOS (NOME_ALUNO);
```

💡 Em PostgreSQL/MySQL, o autoincremento costuma vir de `SERIAL` / `AUTO_INCREMENT` em vez de *sequence* explícita.

Views — consultas com nome

```
CREATE VIEW ALUNOS_MATRICULADOS AS
  SELECT A.NOME_ALUNO AS ALUNO, AO.SEMESTRE,
         O.DIA_SEMANA, P.NOME_PROFESSOR AS PROFESSOR,
         D.NOME_DISCIPLINA
  FROM ALUNOS_OFERTAS AO
  JOIN ALUNOS      A ON AO.MATRICULA_ALUNO = A.MATRICULA_ALUNO
  JOIN OFERTAS     O ON AO.CODIGO_OFERTA   = O.CODIGO_OFERTA
  JOIN PROFESSORES P ON O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR
  JOIN DISCIPLINAS D ON O.CODIGO_DISCIPLINA  = D.CODIGO_DISCIPLINA;
```

Depois é só `SELECT * FROM ALUNOS_MATRICULADOS;` — a complexidade fica escondida.

DML — manipulando os dados

INSERT

```
INSERT INTO ALUNOS
VALUES (40001, 'JOÃO GABRIEL', TO_DATE('02/04/1985', 'DD/MM/YYYY'));

-- forma explícita (recomendada): lista as colunas
INSERT INTO ALUNOS (MATRICULA_ALUNO, NOME_ALUNO, DATA_NASCIMENTO)
VALUES (40002, 'JOÃO JOSÉ', TO_DATE('31/12/2001', 'DD/MM/YYYY'));
```

💡 Listar as colunas deixa o comando **imune** a mudanças na ordem/quantidade de colunas da tabela.

UPDATE

```
-- sempre com WHERE! (senão altera a tabela inteira)
UPDATE ALUNOS
  SET DATA_NASCIMENTO = TO_DATE('29/04/2011','DD/MM/YYYY')
  WHERE MATRICULA_ALUNO = 40004;

-- atualização condicional por conjunto
UPDATE PROFESSORES
  SET FORMACAO = 'PHD'
  WHERE FORMACAO = 'DOUTORADO';
```

Um `UPDATE` sem `WHERE` altera **todas** as linhas. Confira o filtro com um `SELECT` antes.

DELETE

```
DELETE FROM DISCIPLINAS
WHERE CODIGO_DISCIPLINA IN (85853, 75189);

-- apagar professores que não têm nenhuma oferta
DELETE FROM PROFESSORES P
WHERE NOT EXISTS (SELECT 1 FROM OFERTAS O
                  WHERE O.MATRICULA_PROFESSOR = P.MATRICULA_PROFESSOR);
```

💡 Um DELETE só se torna **definitivo** ao confirmar a transação — como desfazer/confirmar é assunto da **Unidade 6**.

DQL — consultando com SELECT

SELECT: projeção, alias e funções

```
SELECT MATRICULA_ALUNO,  
       INITCAP(NOME_ALUNO) AS NOME_ALUNO,  
       TO_CHAR(DATA_NASCIMENTO, 'DD/MM/YYYY') AS NASCIMENTO  
FROM ALUNOS  
WHERE DATA_NASCIMENTO <= TO_DATE('01/01/2000', 'DD/MM/YYYY');
```

- **Projeção:** escolher **colunas** (em vez de `SELECT *`).
- **Alias** (`AS`): renomear a coluna no resultado.
- **Funções:** `INITCAP`, `SUBSTR`, `TO_CHAR`, `UPPER` / `LOWER` ...

WHERE: operadores de filtro

```
WHERE PED.VALOR_TOTAL NOT BETWEEN 100 AND 5000      -- faixa
WHERE CLI.UF IN ('ES','MG')                          -- lista
WHERE CLI.UF NOT IN ('RJ','SP')
WHERE CODIGO_DISCIPLINA_DEPENDENCIA IS NULL          -- ausência de valor
WHERE CLI.CODIGO_CLIENTE < 5 OR CLI.CODIGO_CLIENTE > 25
```



NULL não é igual a nada — nem a NULL. Teste sempre com IS NULL / IS NOT NULL.

LIKE — padrões de texto

- `%` → qualquer sequência de caracteres · `_` → **um** caractere

```
WHERE NOME_ALUNO LIKE '%ANTONIO%'      -- contém ANTONIO
WHERE UPPER(NOME_PRODUTO) LIKE 'MA_____' -- MA + 6 caracteres
WHERE UPPER(NOME_PRODUTO) LIKE '___ACA%' -- 'ACA' na 3ª posição
WHERE UPPER(NOME_PRODUTO) LIKE '%A\_P%' ESCAPE '\' -- '_' literal
```

💡 Use `ESCAPE` quando precisar procurar os próprios caracteres `%` ou `_`.

ORDER BY e DISTINCT

```
-- valores únicos, ordenados
SELECT DISTINCT CIDADE, UF, CEP
  FROM CLIENTES
 ORDER BY UF;

-- ordenação decrescente pelo alias
SELECT NOME_CLIENTE, SUM(VLOR_TOTAL) AS TOTAL
  FROM ...
 ORDER BY TOTAL DESC;
```

DISTINCT remove linhas repetidas; **ORDER BY** ordena (**ASC** padrão, **DESC** inverte).

JOIN — combinando tabelas

```
-- INNER JOIN: só o que casa dos dois lados
SELECT A.NOME_ALUNO, T.TELEFONE
  FROM ALUNOS A
  INNER JOIN TELEFONES_ALUNOS T
    ON A.MATRICULA_ALUNO = T.MATRICULA_ALUNO;

-- LEFT OUTER JOIN: produtos que nunca foram pedidos
SELECT PRO.*
  FROM PRODUTOS PRO
  LEFT OUTER JOIN ITENS_PEDIDOS ITE
    ON PRO.CODIGO_PRODUTO = ITE.CODIGO_PRODUTO
 WHERE ITE.CODIGO_PRODUTO IS NULL;
```

Funções de agregação

```
SELECT MIN(VLOR_TOTAL) AS MINIMO,  
       MAX(VLOR_TOTAL) AS MAXIMO,  
       SUM(VLOR_TOTAL) AS TOTAL,  
       ROUND(AVG(VLOR_TOTAL), 2) AS MEDIA,  
       COUNT(1) AS QTDE  
FROM PEDIDOS;
```

COUNT, SUM, MIN, MAX, AVG **resumem** um conjunto de linhas em um valor.

GROUP BY e HAVING

```
SELECT CLI.CODIGO_CLIENTE,  
       ROUND(AVG(PED.VALOR_TOTAL),2) AS MEDIA_POR_CLIENTE  
FROM CLIENTES CLI  
INNER JOIN PEDIDOS PED  
      ON CLI.CODIGO_CLIENTE = PED.CODIGO_CLIENTE  
GROUP BY CLI.CODIGO_CLIENTE  
HAVING ROUND(AVG(PED.VALOR_TOTAL),2) > 8000  
ORDER BY MEDIA_POR_CLIENTE;
```

- **GROUP BY** agrupa as linhas antes de agregar.
- **HAVING** filtra **grupos** (o **WHERE** filtra **linhas**, antes do agrupamento).

Subconsultas

-- escalar: comparar com a média geral

```
SELECT NOME_PRODUTO, PRECO_PRODUTO
FROM PRODUTOS
WHERE PRECO_PRODUTO > (SELECT AVG(PRECO_PRODUTO) FROM PRODUTOS);
```

-- diferença de conjuntos com IN / NOT IN

```
SELECT CLI.*
FROM CLIENTES CLI
WHERE CLI.CODIGO_CLIENTE IN (SELECT CODIGO_CLIENTE FROM PEDIDOS WHERE DATA_PEDIDO = ...)
AND CLI.CODIGO_CLIENTE NOT IN (SELECT CODIGO_CLIENTE FROM PEDIDOS WHERE DATA_PEDIDO = ...);
```

💡 Uma subconsulta pode devolver um **valor** (escalar), uma **lista** (IN) ou existir/não existir (EXISTS).

Portabilidade entre SGBDs

Recurso	Oracle	PostgreSQL	MySQL
Texto	<code>VARCHAR2</code>	<code>VARCHAR</code>	<code>VARCHAR</code>
Data literal	<code>TO_DATE('..','DD/MM/YYYY')</code>	<code>DATE '2026-01-01'</code>	<code>STR_TO_DATE / '2026-01-01'</code>
Data/hora atual	<code>SYSDATE</code>	<code>NOW()</code>	<code>NOW()</code>
Autoincremento	<code>SEQUENCE</code>	<code>SERIAL</code>	<code>AUTO_INCREMENT</code>
Trata <code>NULL</code>	<code>NVL(x,y)</code>	<code>COALESCE(x,y)</code>	<code>IFNULL / COALESCE</code>
Limitar linhas	<code>FETCH FIRST n ROWS</code>	<code>LIMIT n</code>	<code>LIMIT n</code>

Para praticar

- Você vai exercitar DDL, DML e DQL no **Roteiro Prático de SQL**, em partes:
 - **Parte 1** — DDL (montar o esquema acadêmico)
 - **Parte 2** — DML (insert/update/delete + consultas)
 - **Parte 3** — DQL avançado (domínio de vendas)

 O roteiro é trabalhado **em aula**, na **VM LabDatabase**. O enunciado é disponibilizado pelo professor no AVA.

Bibliografia

- ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7ª ed. Pearson, 2019.
- DATE, C. J. **Introdução a Sistemas de Banco de Dados**. 8ª ed. Elsevier, 2004.
- SILBERSCHATZ, A.; KORTH, H.; SUDARSHAN, S. **Sistema de Banco de Dados**. 7ª ed. Elsevier, 2020.

Dúvidas?

howard.cruz@faesa.br

Próximo →
Procedimentos Armazenados